

I'll review the repository and compare the GDELT, IU HPC, Jetstream2, and AWS options, focusing on practical architecture guidance and concrete meeting talking points.

Thought for 7m 39s ›

Executive recommendation

Keep **model training, embedding experiments, and heavier batch analytics on IU HPC for now**, because you have not yet hit meaningful compute or storage bottlenecks and IU explicitly provides supercomputing plus research storage for high-end, data-intensive research. Move the **GDELT ingestion + article-fetching/web-scraping layer off the shared HPC environment** into a small, controlled, containerized service. My suggested path is:

Now: HPC for modeling + Jetstream2 pilot for GDELT/article ingestion.

Later: AWS only when you need managed orchestration, elastic crawler workers, durable partner-facing storage, production monitoring, or stronger access-control/sharing workflows.

The core architectural point is to separate the pipeline into two planes:

```
Acquisition plane:
GDELT files -> URL extraction -> crawl queue -> polite article fetcher -> bronze storage

Modeling plane:
bronze -> silver TextRecord -> gold embeddings/features -> transformer forecasts -> partner outputs
```

That separation lets you keep GPUs and SLURM where they are useful, while moving outbound network scraping away from a shared HPC IP space.

What I found in the repository

The repository is already pointed in the right direction. It describes itself as a pipeline for ingesting and processing energy-related textual data for NLP-driven load forecasting, with current sources including MISO notifications, NewsAPI articles, and National Weather Service alerts. It also exposes CLI commands for ingestion, standardization, and embedding generation, and its README says embeddings use NVIDIA NeMo Retriever NIM embedding models.  [GitHub](#)

The project documentation already defines a **medallion-style architecture**: Bronze for raw source data, Silver for standardized/cleaned records, and Gold for modeling-ready artifacts such as embeddings. That is the right conceptual foundation for GDELТ because you will need reproducible raw capture, reprocessing, source traceability, and incremental updates.  [GitHub](#)

The code has a useful source abstraction: `TextSource` defines `fetch_raw`, `parse_records`, `standardize_record`, schema validation, and deterministic record IDs. That is a good pattern for MISO, weather alerts, and small API sources. However, the current `run()` implementation fetches all raw data, parses all records, converts everything to a list, then builds a pandas DataFrame. That pattern is risky for bulk GDELТ ingestion because GDELТ backfills and URL/article collection should be handled as streaming, chunked, restartable jobs rather than one in-memory source run.  [GitHub](#)

The repository already has separate pipeline modules for backfill, embedding, ingestion, and standardization, and existing source modules for IEM AFOS, MISO, NewsAPI, and weather alerts. I did not see a GDELТ source module in the public source listing yet, so I would treat GDELТ as the first “large-file plus crawl queue” source rather than just another small API adapter.  [GitHub +1](#)

One practical issue: the current `requirements.txt` appears to mix ingestion dependencies with notebook/dev packages and GPU/modeling packages, including CUDA-related packages, PyTorch, Transformers, sentence-transformers, and an editable Git install of the repository. For production-style ingestion workers, split this into separate dependency sets such as `requirements-ingest.txt`, `requirements-model.txt`, and `requirements-dev.txt`, or use optional dependency groups in `pyproject.toml`.  [GitHub](#)

GDELТ-specific advice

Your shift away from repeatedly querying the GDELТ API and toward downloading/filtering GDELТ files directly is sensible for historical ingestion. GDELТ says its global event database, GKG, and related data streams update every 15 minutes; it also makes raw data files available for download and analysis.

 [GDELТ Pro... +1](#)

Use GDELТ as a **metadata and URL discovery layer**, not as the final article-text layer. A robust GDELТ workflow should look like this:

1. Download GDELТ update/backfill files.
2. Store the original file or file metadata in bronze.
3. Parse only the fields needed for filtering.
4. Filter by topic, geography, time, source language, domain, event category, etc.
5. Extract candidate article URLs.
6. Canonicalize and deduplicate URLs.
7. Send unique URLs into a crawl queue.
8. Fetch article pages politely and slowly, with per-domain throttles.
9. Store article HTML/text, fetch metadata, status codes, and parser versions.
10. Convert to silver TextRecord only after validation.

For GDELТ, I would add a separate source family rather than forcing everything through the current small-source `TextSource.run()` pattern:

```
GdelтFileSource
- downloads/parses GDELТ files
- emits candidate records and URLs

ArticleFetchSource
- fetches article URLs
- enforces robots/ToS/rate-limit policy
- records HTTP status, retries, content hash, parser version

GdelтStandardizer
- maps GDELТ metadata + article text into TextRecord
```

The important design principle is **idempotency**: if a job fails halfway through a month of GDELT history, you should be able to rerun it without duplicating files, refetching already-successful URLs, or corrupting silver/gold outputs.

Web scraping: is HPC appropriate?

For small, occasional downloads, HPC can be fine. For large-scale, highly parallel scraping, a shared HPC cluster is often the wrong place because the risk is not mainly CPU load; it is shared network reputation, policy exposure, and impact on other users. Harvard’s research computing guidance, for example, strongly discourages using its cluster for scraping and explicitly warns that highly parallel scraping can get a cluster or university IP range blacklisted, affecting other researchers. [docs.rc.fas.har...](#)

IU’s appropriate-use policy also matters: IU IT resources must be used for university mission purposes, must be lawful, and units may impose more restrictive local policies than the university-wide baseline. That supports the IU HPC team’s concern: even if the research purpose is valid, the HPC service may still reasonably restrict scraping behavior to protect shared infrastructure. [University Poli...](#)

For your project, I would phrase the position this way in the AWS/Jetstream2 discussion:

“We do not need cloud because HPC is failing computationally. We need a better home for the outbound acquisition layer, because crawling many external news sites from a shared HPC environment can create policy, rate-limit, and IP-reputation risks.”

That is a much stronger framing than “Should we move everything to AWS?”

HPC vs Jetstream2 vs AWS

Option	Best use in your project	Main advantages	Main concern
IU HPC / SLURM	Embeddings, transformer training, batch feature generation, large analytical jobs	Already working; strong for compute-heavy and GPU/AI research; close to IU research storage services such as SDA, Geode, and Slate. uits.iu....	Not ideal for I always-on cr network servi
Jetstream2	First home for scheduled GDELT ingestion and article fetching	Designed as a cloud environment for researchers; supports always-on research infrastructure; CPU VMs have no runtime restrictions while the allocation and SUs remain active; default allocations include 1 TB of Jetstream2 storage. docs.jetstream-... +2	Not primarily backup need functions for i path. GitHu
AWS	Productionized ingestion, elastic backfills, managed queues, partner-facing storage/access, observability	S3, Batch, ECS/Fargate, Step Functions, EventBridge, SQS, Glue/Athena, IAM, budget controls, logs, lifecycle policies. EventBridge can schedule AWS Batch jobs, and Step Functions supports retries/backoff/error handling. AWS Documen... +1	Cost, IAM cor and risk of ov S3 costs incl replication, ar

My recommendation: **Jetstream2 first, AWS second**. Jetstream2 is the better immediate fit if the main question is “where should scheduled academic ingestion and scraping live?” AWS becomes more compelling when you need managed elasticity, partner delivery, stronger access controls, and lower-maintenance orchestration.

Suggested near-term architecture

1. Keep the lake format cloud-portable

Do not bind the data layout too tightly to HPC paths, Jetstream2 volumes, or S3. Use a storage abstraction and write paths that can map cleanly to local files, Slate/SDA, Jetstream2 volumes, or S3:

```
bronze/  
  gdelit/  
    table=events/  
    table=mentions/  
    table=gkg/  
  articles/  
    source_domain=example.com/
```

```

    fetch_date=YYYY-MM-DD/

silver/
  text_records/
    source=gdelta/
    published_date=YYYY-MM-DD/

gold/
  embeddings/
    model=...
    published_date=YYYY-MM-DD/

manifests/
  runs/
  gdelta_files/
  url_fetch_state/

```

For now, Parquet is enough. If the dataset becomes multi-terabyte or you need schema evolution, partition pruning, and SQL-style querying across many runs, consider Apache Iceberg. AWS Glue supports Apache Iceberg tables in S3 and can integrate them with the Glue Data Catalog. [AWS Document...](#)

2. Add a durable run manifest

Every ingestion run should write a manifest like:

```

<> JSON
{
  "run_id": "2026-06-10T140000Z",
  "source": "gdelta",
  "job_type": "gdelta_file_download",
  "code_commit": "...",
  "config_hash": "...",
  "input_files": [...],
  "output_paths": [...],
  "records_read": 123456,
  "records_written": 4567,
  "urls_discovered": 3210,
  "urls_new": 1200,
  "errors": 17,
  "started_at": "...",
  "finished_at": "..."
}

```

This is more important than cloud choice. Without manifests, cloud simply makes a fragile workflow more expensive.

3. Add a URL state table

For article fetching, maintain a durable table with at least:

```

canonical_url
url_hash
source_url
source_domain
gdelta_source_file
first_seen_at
last_attempt_at
next_allowed_attempt_at
status
http_status
num_attempts
content_hash
article_storage_path
parser_version
error_message

```

This lets you pause, resume, deduplicate, retry, and prove what happened.

4. Use per-domain throttling

Do not just set "N workers" globally. Use **per-domain** budgets:

```

<> YAML

```

```
crawler:
  global_max_workers: 8
  per_domain_max_concurrency: 1
  default_delay_seconds: 5
  timeout_seconds: 20
  max_retries: 3
  backoff:
    initial_seconds: 30
    multiplier: 2
    jitter: true
  respect_robots_txt: true
  max_response_bytes: 5000000
```

The dangerous scenario is not one polite worker; it is many independent workers each thinking they are polite while collectively hammering the same publisher.

A good Jetstream2 pilot

A practical first deployment could be very simple:

```
Jetstream2 CPU VM
  Docker / Docker Compose
    eml_transformer-ingest container
    small Postgres or SQLite state DB
    optional Redis/SQS-like queue
  systemd timer or cron
    run GDELT file ingestion every 15/30/60 minutes
    run article fetcher with low concurrency
  attached volume
    temporary bronze/silver outputs
  nightly transfer
    sync accepted outputs to IU storage / HPC-facing storage
```

Use Jetstream2 for the acquisition plane only. Continue using IU HPC/SLURM for embedding generation and forecasting jobs until there is a concrete reason to move those workloads.

The key operational rule: Jetstream2 volumes should not be the only copy of important data. Jetstream2 documentation says projects receive default storage, but it also notes that automated backups are not provided for volumes, so you should back up or export data regularly. [docs.jetstream... +1](#)

When AWS becomes worth it

AWS is probably not necessary just because you are using GDELT. It becomes worth considering when at least two or three of these become true:

You need to run many crawler workers for historical backfills and finish in days rather than weeks.

You need durable, centrally managed object storage with lifecycle rules, cross-team access control, and auditability. S3 lifecycle policies can transition data to lower-cost storage classes over time, which is useful for bronze archives that are rarely reread. [AWS Documenten... +1](#)

You need managed scheduling and retry logic instead of hand-maintained cron/SLURM scripts. EventBridge can schedule AWS Batch jobs, and Step Functions provides native retry/backoff/error handling for workflows. [AWS Documenten... +1](#)

You need to share forecast outputs with industry partners through controlled access mechanisms rather than copying files manually.

You need CloudWatch-style logging/metrics/alarms, budget alerts, IAM isolation, and clearer operational ownership.

You need SQL-style querying over a growing data lake using Glue/Athena/Iceberg rather than only local Parquet files. [AWS Documenten... +1](#)

A reasonable AWS version would be:

```
EventBridge schedule
  -> Step Functions workflow
    -> AWS Batch / ECS Fargate: download GDELT files
```

```
-> S3 bronze
-> SQS queue of candidate URLs
-> AWS Batch / ECS Fargate: article fetch workers
-> S3 bronze/articles
-> Glue/Athena/Iceberg silver tables
-> HPC or AWS GPU job for embeddings
-> S3 gold forecasts
-> partner access layer
```

I would avoid Lambda for full article scraping unless the pages are simple and the dependencies are light. Batch, ECS/Fargate, or EC2 are better for crawling because they handle longer runtimes, heavier dependencies, and more predictable resource control.

Code changes I would prioritize before any cloud migration

1. **Add GDELT as a streaming/chunked source, not a pandas-all-at-once source.** The existing `TextSource.run()` pattern is clean, but it builds a full DataFrame after fetching and parsing all records. That should not be the core pattern for GDELT backfills. [GitHub](#)
2. **Split ingestion and modeling dependencies.** Ingestion workers should not install CUDA, PyTorch, Transformers, notebook tooling, and embedding libraries unless they need them. The current requirements file mixes lightweight ingestion packages with the GPU/modeling stack. [GitHub](#)
3. **Add stateful manifests and checkpoints.** Track GDELT files downloaded, checksums, row counts, URL counts, fetch attempts, and output paths.
4. **Add a crawler policy module.** Make robots/ToS checks, per-domain throttling, retries, max response size, user-agent, timeout, and backoff explicit in config.
5. **Add tests before scaling.** I did not see a `tests/` directory in the public repository root listing. At minimum, add tests for URL canonicalization, deduplication, GDELT row parsing, schema validation, and idempotent reruns. [GitHub](#)
6. **Add structured logging.** Each job should log `run_id`, source, input file, URL/domain, status code, retry count, output path, and record counts.
7. **Add a rights/sharing policy.** GDELT data itself is open, but full publisher article text is a different issue. For partner sharing, the safer default is to share derived forecasts, signals, embeddings, metadata, or aggregate features rather than raw article bodies unless permissions are clear. GDELT makes its own data files available, but external websites' robots.txt, terms, and AI/scraping policies still matter.

[GDELT Pro... +1](#)

What to ask in the AWS / cloud meeting

Use the meeting to avoid overbuilding. I would ask:

For AWS:

What would the monthly cost look like for S3 storage, requests, data transfer, Batch/Fargate compute, CloudWatch logs, and Glue/Athena queries at three scales: pilot, expected, and worst-case backfill? AWS S3 pricing depends on storage, requests, retrieval, transfer, management/analytics, replication, and query/transform usage, so ask for a concrete estimate rather than a generic "S3 is cheap" answer.

[Amazon Web S...](#)

For AWS:

Would they recommend Batch, ECS/Fargate, or EC2 Spot for polite scraping workers? Ask specifically how they would implement per-domain throttling across many workers.

For AWS:

How should partner delivery be handled: S3 bucket policies, presigned URLs, IAM roles, API Gateway, QuickSight, or a separate portal?

For IU / Jetstream2:

Is scheduled outbound article fetching acceptable on Jetstream2 under your allocation? Jetstream2 is positioned as an always-on research cloud environment, but you still want written confirmation for web scraping workloads. [docs.jetstream-... +1](#)

For IU HPC:

Is direct GDELT file download and filtering acceptable on HPC if article fetching is moved elsewhere? This

distinction matters: downloading known public data files is different from parallel scraping of many third-party news sites.

For everyone:

Where should the authoritative copy of bronze/silver/gold data live: IU Slate/SDA, Jetstream2 volume, S3, or a hybrid?

Suggested stance for your team

Do **not** frame this as “Should we move to AWS?” Frame it as:

“Our modeling pipeline is currently fine on IU HPC. The architectural question is whether the outbound acquisition layer—GDELT file ingestion, URL extraction, and article fetching—should run somewhere other than a shared HPC environment. We want a setup that is reproducible, polite to external sites, restartable, and portable across IU HPC, Jetstream2, and AWS.”

That position is technically sound and should resonate with both the HPC team and AWS/cloud advisors.

HOW WOULD AWS HELP THIS

I'll focus on the concrete ways AWS could help this pipeline—especially scraping risk, scheduling, scaling, monitoring, storage, and partner delivery—while separating real benefits from unnecessary complexity.

Thought for 2m 31s >

AWS would help **less with the modeling right now** and **more with the data acquisition/infrastructure side**.

For your project, the main AWS value is this:

Move GDELT ingestion, article URL queues, article scraping, storage, retry logic, monitoring, and partner-facing outputs into managed cloud infrastructure, while keeping heavy model training on IU HPC until HPC stops being sufficient.

1. AWS gives you a better home for web scraping than shared HPC

The strongest reason to use AWS is **not compute**. It is that large-scale outbound web scraping is awkward and potentially risky on shared HPC infrastructure.

On AWS, you can run scraping jobs in an isolated account/VPC, control concurrency, control IP exposure, log every request, and shut workers down when they are done. That makes the scraping layer more operationally contained than running many outbound requests from IU's shared HPC environment.

But AWS is **not a license to scrape aggressively**. AWS's acceptable-use rules prohibit illegal, abusive, or rights-violating activity, and AWS users are responsible for what they run. So AWS helps you build a compliant, throttled crawler; it should not be used to evade robots.txt, paywalls, rate limits, or publisher restrictions. [Amazon Web S...](#)

The right framing is:

HPC: modeling, embeddings, batch analytics

AWS: GDELT ingestion, URL queueing, article fetching, storage, monitoring, partner delivery



2. AWS makes scheduled ingestion cleaner than SLURM cron-style jobs

Right now, you are using scheduled SLURM jobs. That works, but SLURM is primarily a batch scheduler for compute jobs, not a full ingestion orchestration system.

AWS could replace the recurring ingestion schedule with **EventBridge Scheduler**, which supports cron-style and rate-based recurring schedules from a managed service. [AWS Documen...](#)

For example:

```
Every 15 minutes:
EventBridge Scheduler
-> start GDELT download job
-> parse GDELT files
```



```
-> extract URLs
-> enqueue URLs for article fetching
```

This is useful because GDELT itself updates frequently, and your pipeline will eventually need regular ingestion rather than one-off backfills.

3. AWS can run containerized ingestion jobs without maintaining servers

Your ingestion code could be packaged as a Docker container and run through **AWS Batch**, **ECS/Fargate**, or EC2. AWS Batch is specifically designed to plan, schedule, and run containerized batch workloads across AWS compute options such as ECS, EKS, Fargate, Spot, and On-Demand instances. [AWS Document...](#)

That means you could define jobs like:

```
gdelt-download-job
gdelt-parse-job
url-dedup-job
article-fetch-job
article-cleaning-job
silver-standardization-job
```

Then AWS starts compute only when needed and stops it afterward. This matters for historical GDELT backfills because you may sometimes need many workers for a short period, then almost none afterward.

AWS Batch jobs can also be triggered by EventBridge, so recurring ingestion can be automated without keeping a VM running all the time. [AWS Document...](#)

4. AWS gives you proper queues for article URLs

The GDELT workflow naturally becomes a queueing problem:

```
GDELT record -> article URL -> fetch attempt -> success/failure/retry
```

AWS would help by using **Amazon SQS** as a durable queue of article URLs. Instead of one script trying to scrape everything, you would push URLs into a queue and let workers pull from it.

That gives you:

```
URL discovered
-> add to SQS queue
-> crawler worker fetches article
-> success: save article text/HTML
-> failure: retry later
-> repeated failure: send to dead-letter queue
```

SQS supports dead-letter queues, which are useful for isolating messages that repeatedly fail so you can debug them instead of silently losing them or crashing the whole job. [AWS Document...](#)

This is a big upgrade over a simple script because web scraping failures are normal: 403s, 404s, 429s, timeouts, broken redirects, bot blocks, paywalls, bad encodings, and parser failures.

5. AWS helps with retries, backoff, and failure handling

Your pipeline has multiple fragile steps:

```
download GDELT file
parse file
extract URLs
deduplicate URLs
fetch articles
extract article text
write bronze/silver data
generate embeddings
produce forecasts
```

AWS Step Functions can orchestrate those steps and define what happens when a step fails. Step Functions supports retry policies, catch logic, fallback states, and error handling. [AWS Document...](#)

For your project, that means you could explicitly encode logic like:


```

If GDELT download fails:
    retry 3 times with backoff

If article fetch returns 429:
    delay and retry later

If article parser fails:
    store raw HTML and send URL to review queue

If embedding job fails:
    do not mark run as complete

If full run succeeds:
    write manifest and publish outputs

```

That is much cleaner than burying the retry logic inside ad hoc Python scripts.

6. AWS gives you a scalable data lake for bronze/silver/gold storage

Your medallion architecture maps very naturally to **S3**:

```

s3://project-data/bronze/gdelt/
s3://project-data/bronze/articles/
s3://project-data/silver/text_records/
s3://project-data/gold/embeddings/
s3://project-data/gold/forecasts/
s3://project-data/manifests/

```

Amazon S3 is designed to store and retrieve large amounts of data, and it is commonly used as the storage layer for data lakes. [AWS Document...](#)

The main benefit is not just “more storage.” It is that S3 gives you:

```

durable object storage
versioned raw data
clear separation of bronze/silver/gold layers
lifecycle policies
access control
partner sharing options
query integration

```

For example, raw GDELT files and raw article HTML could live in bronze, cleaned standardized records in silver, and embeddings/forecast outputs in gold.

AWS S3 Lifecycle rules can automatically move older or infrequently accessed objects to cheaper storage classes or expire temporary objects. [AWS Document...](#)

That matters because bronze data can grow quickly and may not need to stay in expensive hot storage forever.

7. AWS makes backfills easier

Historical GDELT ingestion is different from routine daily ingestion.

Routine ingestion might be:

```

Every 15 minutes or every hour:
    download latest GDELT files
    extract URLs
    fetch new articles slowly

```

Backfill ingestion might be:

```

For several years of historical data:
    process thousands of files
    extract millions of URLs
    deduplicate aggressively
    fetch only relevant articles

```

AWS helps because you can temporarily scale out for backfills:

```
Small daily mode:
  1-2 workers

Backfill mode:
  20-100 workers, with strict per-domain throttling

After backfill:
  scale back down
```

HPC can also scale compute, but AWS is better suited for elastic, network-heavy, queue-driven jobs where workers may need to run independently and retry over time.

8. AWS gives you better monitoring and auditability

With AWS, each ingestion job can write logs and metrics to **CloudWatch**. CloudWatch is AWS's monitoring service for logs, metrics, alarms, dashboards, and application/infrastructure visibility. [AWS Document...](#)

For your project, you would want metrics like:

```
GDELT files downloaded
GDELT records parsed
URLs discovered
URLs deduplicated
articles fetched successfully
HTTP 403 count
HTTP 404 count
HTTP 429 count
timeout count
parser failure count
bytes stored
cost per run
duration per run
```

That helps answer operational questions:

```
Did last night's ingestion run?
How many URLs failed?
Which domains are rate-limiting us?
Are we accidentally fetching too aggressively?
How much did the backfill cost?
Which code version produced this dataset?
```

This is one of the main places where AWS is better than a loose collection of SLURM scripts.

9. AWS helps with partner-facing forecast outputs

Eventually, you want to share forecast outputs with industry partners. AWS is useful here because S3, IAM, bucket policies, and presigned URLs give you controlled ways to share data.

By default, S3 objects are private, and object owners can share selected objects using presigned URLs. [AWS Document...](#)

For example:

```
gold/forecasts/partner_a/latest.parquet
gold/forecasts/partner_a/latest.csv
gold/forecasts/partner_a/dashboard_export.json
```

You could give partners access to only the outputs they are allowed to see, rather than giving access to raw scraped text, embeddings, or internal intermediate data.

This is important because raw article text may raise copyright/licensing issues, while forecast outputs, metadata, and aggregate features are usually easier to share.

10. AWS helps enforce access control

AWS IAM lets you define which users, roles, or services can access which resources. IAM policies determine whether a request is allowed or denied, and AWS recommends moving toward least-privilege permissions. [AWS Document... +1](#)

For your project, you could separate permissions like this:

```

ingestion-worker role:
  can write bronze data
  cannot read partner outputs

modeling-worker role:
  can read silver data
  can write gold embeddings

partner-output role:
  can read only gold/forecasts/partner_x/

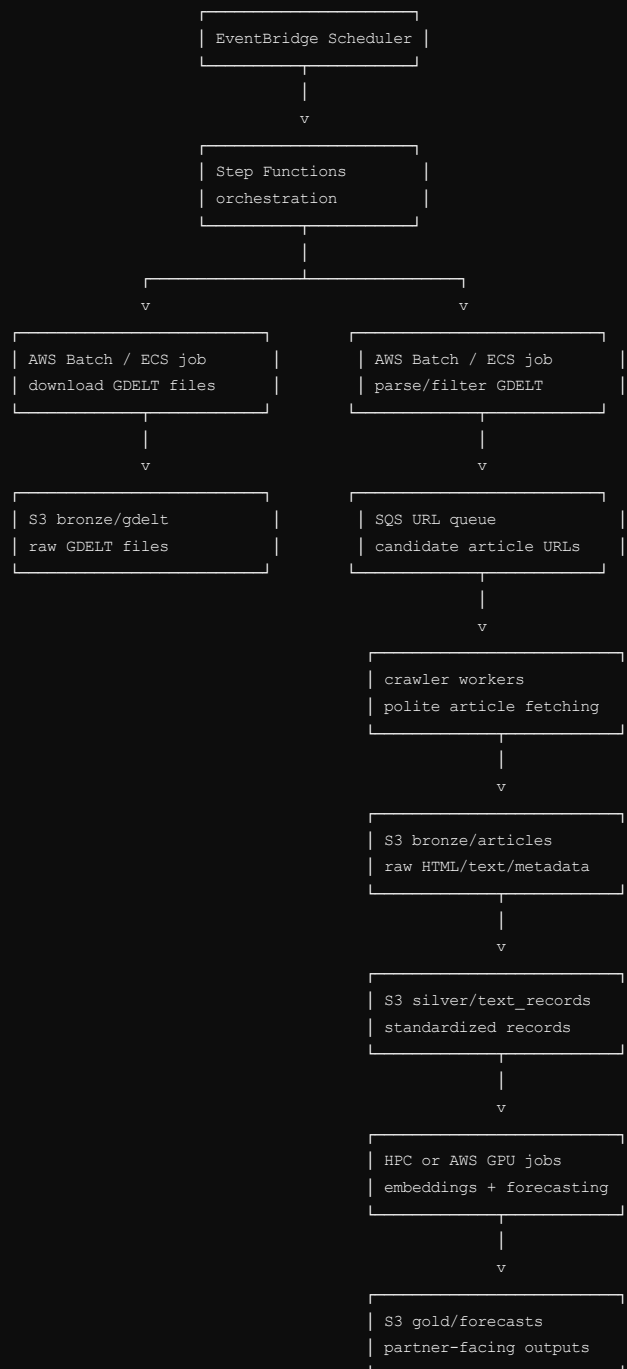
admin role:
  can manage infrastructure

student/researcher role:
  can inspect logs and selected datasets

```

That is much cleaner than managing access through shared filesystem conventions alone.

What an AWS version of your pipeline could look like



I would **not** move everything to AWS immediately. I would move only the acquisition layer first.

Where AWS helps most for your exact project

Problem	How AWS helps
HPC team is concerned about scraping	Move outbound article collection away from shared HPC infrastructure.
GDELT creates many files and URLs	Use S3 for raw files and SQS for durable URL queues.
Scraping has many partial failures	Use retry logic, dead-letter queues, and run manifests.
Backfills may need temporary scale	Use AWS Batch/ECS workers that scale up and shut down afterward.
SLURM is okay but not ideal for ingestion workflows	Use EventBridge schedules and Step Functions orchestration.
Dataset will grow over time	Use S3 bronze/silver/gold layout plus lifecycle rules.
Need to know what happened in each run	Use CloudWatch logs, metrics, dashboards, and alarms.
Need to share outputs with industry partners	Use S3, IAM, bucket policies, and presigned URLs for controlled access.

Where AWS does not help much yet

AWS probably does **not** help much with the transformer modeling side right now, because you said IU HPC is already handling compute and storage successfully.

AWS would also not automatically solve:

```
publisher robots.txt restrictions
article copyright/licensing questions
HTTP 403/429 responses
bad article extraction quality
duplicate URLs
paywalls
model quality
data leakage
forecast validation
```



Those are pipeline design and governance issues, not cloud issues.

AWS also adds operational burden:

```
cost management
IAM complexity
cloud security configuration
data transfer costs
infrastructure-as-code maintenance
monitoring setup
```



So the case for AWS should be based on operational need, not just “cloud is modern.”

My practical recommendation

Use AWS only where it clearly improves the project:

```
Phase 1:
Keep modeling on IU HPC.
Pilot AWS for GDELT ingestion + URL queue + polite article fetching.

Phase 2:
```



```
Store bronze/silver/gold outputs in S3 or sync from AWS to IU storage.  
Add run manifests, CloudWatch logs, and dead-letter queues.
```

Phase 3:

Use AWS for partner-facing forecast delivery.

Only consider AWS GPUs if HPC becomes a bottleneck or partner delivery requires a fully cloud-hosted

The best one-sentence answer is:

AWS would help by turning the fragile, network-heavy GDELT/article-collection part of the project into a scalable, monitored, restartable, and partner-shareable ingestion system—while letting IU HPC continue doing the modeling work. what methods do i use to build the IaC? in order of preference

I'm checking the current AWS-native options and the main IaC frameworks so I can give you a sensible ordering instead of a generic list.

For **your** project, I would rank the IaC methods like this:

1. **AWS CDK in Python** — my first choice. It fits a Python-heavy team, lets you define AWS infrastructure in a familiar programming language, and then synthesizes CloudFormation under the hood. AWS also describes it as a good fit for teams with strong software engineering skills and for rapid iteration/reuse.  AWS Documenten... +1
2. **Terraform** — my second choice. It is a strong option if you want broader portability, a mature module/state workflow, or think you may want non-AWS resources later. Terraform is declarative, manages state, and has a broad provider ecosystem.  HashiCorp Dev... +1
3. **CloudFormation directly** — my third choice. It is the most AWS-native option, with explicit JSON/YAML templates, managed stacks, change sets, rollback, drift detection, and stack sets. I would pick this if you want maximum AWS alignment and minimal abstraction.  AWS Documenten... +1
4. **AWS SAM** — only if the pipeline becomes mostly serverless. AWS describes SAM as a CloudFormation extension optimized for Lambda, API Gateway, and Step Functions. That makes it useful for a serverless ingestion layer, but less compelling as the main IaC approach for your whole stack.  AWS Documenten...

If you want one recommendation to start with: **use AWS CDK in Python, modularized into separate stacks for networking, ingestion, storage, and monitoring.** That gives you code-first development now, while still staying inside AWS's native deployment path through CloudFormation.  AWS Documenten... +1